

LLM Fine-Tuning: SFT and DPO on TinyLlama-1.1B

NLP Assignment Report

Table of Contents

- 1. [Platform Details](#)
 - 2. [Data Details](#)
 - 3. [Baseline Results](#)
 - 4. [SFT Experiments](#)
 - 5. [DPO Experiments](#)
 - 6. [Three-Way Comparison](#)
 - 7. [Analysis and Insights](#)
 - 8. [Reproducibility](#)
 - 9. [References](#)
-

1. Platform Details

All experiments were conducted on **Google Colab** using a T4 GPU (15 GB VRAM).

Component	Details
Platform	Google Colab (free tier)
GPU	NVIDIA Tesla T4, 15 GB VRAM
CPU RAM	~12 GB
Python	3.10
PyTorch	2.4.x (Colab default)
CUDA	12.1

The training pipeline was implemented across three Jupyter notebooks: `00_baseline_evaluation.ipynb`, `01_sft_training.ipynb`, and `02_dpo_training.ipynb`. Model adapters and result CSVs were persisted to Google Drive between sessions.

2. Data Details

2.1 Manual Test Set

A set of 10 prompts was manually constructed to cover diverse task types: factual question answering, instruction following, mathematical reasoning, logical inference, creative writing, summarization, and open-ended analysis. Reference answers were generated using ChatGPT-4o and serve as the gold standard for all BLEU and BERTScore evaluations.

Table 1: Test Prompt Categories

ID	Type	Prompt (truncated)
1	Factual	What is the greenhouse effect...?
2	Factual	Explain the difference between supervised and unsupervised learning...?
3	Instruction	Write a short professional email declining a job offer...
4	Instruction	List five practical tips for improving productivity while working from home.
5	Instruction	Explain the concept of recursion in programming with a simple example.
6	Math reasoning	A store sells apples at \$0.50 each and oranges at \$0.75 each...
7	Logic	If all roses are flowers and some flowers fade quickly...
8	Creative writing	Write a short two-paragraph story about a robot that learns to paint.
9	Summarization	Summarize the main idea: 'The Industrial Revolution began...'
10	Open-ended	What are the key advantages and disadvantages of remote work...?

2.2 SFT Dataset — tats-u-lab/alpaca

Property	Value
Source	HuggingFace Hub: tats-u-lab/alpaca
Full dataset size	52,002 samples
Subset used	10,000 samples

Property	Value
Train split	9,000 (90%)
Validation split	1,000 (10%)
Format	instruction / input / output triplets
Origin	GPT-3 text-davinci-003 generations

Justification: The Alpaca dataset provides clean instruction–response pairs across a wide range of task types, making it well-suited for general instruction-following SFT. A 10,000-sample subset was chosen to keep per-trial training time under 40 minutes on Colab's T4 GPU. The full dataset was not used to preserve the compute budget for 5 trials × 2 notebooks.

Preprocessing: Each sample was formatted into the Alpaca prompt template (see Section 8). Samples with a non-empty `input` field used a two-part instruction+input template; samples without an input used the shorter single-instruction template. The dataset was shuffled with seed 42 before subsetting.

2.3 DPO Dataset — Intel/orca_dpo_pairs

Property	Value
Source	HuggingFace Hub: Intel/orca_dpo_pairs
Full dataset size	~12,859 samples
Subset used	3,000 samples
Train split	~2,700 (90%)
Validation split	~300 (10%)
Format	prompt / chosen / rejected triplets
Origin	Orca-style system prompts with GPT-4 (chosen) and GPT-3.5 (rejected) responses

Justification: The Orca DPO pairs dataset provides high-quality human-preference signals derived from GPT-4 vs. GPT-3.5 responses. A 3,000-sample subset was used; per-trial DPO training is slower than SFT due to the dual forward pass, so a smaller dataset keeps each trial under ~30 minutes. Samples where `chosen == rejected` were filtered out.

3. Baseline Results

Fill this section after running `00_baseline_evaluation.ipynb`.

3.1 Model and Tokenizer

- Model: TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T
- Tokenizer: same checkpoint

- Quantization: 4-bit NF4 (bitsandbytes)
- Generation parameters: max_new_tokens=200, temperature=0.7, top_p=0.9, do_sample=True

3.2 Baseline Metric Results

Table 2: Baseline BLEU and BERTScore per Prompt

ID	Type	BLEU	BERTScore F1
1	Factual	6.101	0.1038
2	Factual	2.3142	0.0057
3	Instruction	4.0813	0.0389
4	Instruction	0.5072	-0.2058
5	Instruction	1.0312	-0.1355
6	Math reasoning	2.3848	-0.1583
7	Logic	0.8597	-0.0633
8	Creative writing	0.6766	-0.2459
9	Summarization	6.0717	0.315
10	Open-ended	1.9722	0.1836
Mean		<i>2.6000</i>	<i>-0.0162</i>

3.3 Observations

The base TinyLlama-1.1B model produces consistently poor outputs across all task types, confirming the expected behaviour of an untuned base model when presented with instruction-style prompts.

- **Prompt continuation instead of answering:** Prompts 1, 2, 5, and 7 caused the model to generate additional `### Instruction: / ### Response:` blocks rather than answering the question — a hallmark failure of a base (non-chat-tuned) model that has learned the training data format but not when to stop.
 - **Severe repetition loops:** Prompts 4 and 8 triggered degenerate repetition (the same phrase repeated 10+ times), yielding near-zero BLEU and the two lowest BERTScore F1 values in the set (−0.2058 and −0.2459).
 - **Surface-form copying:** Prompt 9 (Summarization) achieved the highest BERTScore F1 (0.315) and BLEU (6.07) because the model largely copied the prompt text verbatim, incidentally overlapping with the reference.
 - **Math and logic failures:** Both reasoning prompts (6 and 7) received negative BERTScore F1, indicating semantic divergence worse than a random baseline. The model produced an incorrect formula for the arithmetic problem and completely ignored the logical structure of the syllogism.
-

4. SFT Experiments

Fill this section after running `01_sft_training.ipynb`.

4.1 Training Setup

- Base model: TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T
- Dataset: tatsu-lab/alpaca (10k subset, 90/10 split)
- Fine-tuning method: LoRA (PEFT) via TRL SFTTrainer
- Quantization: 4-bit NF4 during training
- Optimizer: paged AdamW 8-bit
- Shared hyperparameters: `lora_dropout=0.05`, `bias=none`, `warmup_ratio=0.03`, `lr_scheduler=cosine`, `weight_decay=0.01`

Due to Colab session time limits, training was split across three sessions: T1–T2 in the first session, T3 alone in the second (memory-safe config), and T4–T5 in the third. Trial configurations for T3 and T4 were adjusted downward from the original plan (lower rank, smaller batch size, fewer target modules) to avoid CUDA OOM errors on the 15 GB T4 GPU. Evaluation metrics (BLEU, BERTScore F1) were recovered post-training using `01b_sft_eval_results.ipynb`, which loaded each saved adapter from Drive without retraining.

4.2 Trial Configurations and Results

Table 3: SFT Trial Configurations and Evaluation Results

Trial	r	α	Target Modules	LR	Batch	Grad Accum	Epochs	Val Loss	BLEU	BERT Score F1
T1	4	8	q, v	2e-4	4	4	1	1.0083	4.8801	0.1889
T2	8	16	q, v	2e-4	4	4	2	0.9867	6.4503	0.2210
T3	8	16	q, v, o	1e-4	2	8	2	0.9819	5.2721	0.2155
T4	16	32	q, k, v, o, gate, up, down	5e-5	1	16	1	0.9801	5.1978	0.2198
T5	8	16	q, v	5e-4	2	8	1	0.9873	5.6923	0.1872

Bold row = best trial. T3 and T4 configs were adjusted from the original plan due to GPU memory constraints (see §4.1).

4.3 Best SFT Model

Selected Trial: T2 — `r=8`, `lora_alpha=16`, `target_modules=[q_proj, v_proj]`, `lr=2e-4`, 2 epochs

Rationale: T2 achieved the highest BERTScore F1 (0.2210) and the highest BLEU (6.4503) simultaneously, making it the clear winner on both primary and secondary metrics. Doubling the LoRA rank to $r=16$ (T4) or expanding to additional attention modules including the MLP projectors (T4) did not improve scores despite more trainable parameters, suggesting diminishing returns on a 10k-sample dataset. T3 ($r=8 + o_proj$) came close on BERTScore F1 (0.2155) but underperformed on BLEU (5.2721). T5's aggressive learning rate ($5e-4$) with only 1 epoch produced the worst BERTScore F1 (0.1872), confirming that additional epochs matter more than learning rate scaling at this dataset size. T2's combination of moderate rank, 2 epochs, and standard learning rate ($2e-4$) represents the optimal point in the configuration space explored.

4.4 Sample Outputs from Best SFT Model (Trial T2)

Prompt 1 — Factual: *"What is the greenhouse effect and how does it contribute to climate change?"*

Generated: The greenhouse effect is a process in which greenhouse gases in the atmosphere absorb the heat from the sun and keep it trapped inside the Earth's atmosphere. This keeps the Earth's surface temperature at a warmer level...

Reference: The greenhouse effect is a natural process in which certain gases in Earth's atmosphere, such as carbon dioxide, methane, and water vapor, trap heat from the Sun...

Prompt 3 — Instruction: *"Write a short professional email declining a job offer politely."*

Generated: Dear Mr/Ms. X, I am sorry to inform you that I cannot accept your offer at this time. I would like to thank you for your consideration and I wish you the best of luck in your search. Kind regards...

Reference: Subject: Thank You for the Opportunity. Dear [Hiring Manager Name], Thank you very much for offering me the position at your company...

Prompt 5 — Instruction: *"Explain the concept of recursion in programming with a simple example."*

Generated: Recursion is a programming technique in which a program calls itself repeatedly, in order to solve a problem. For example, if a program wants to calculate the sum of all the numbers from 0 to 10, it can call itself recursively...

Reference: Recursion is a programming technique where a function calls itself to solve a smaller version of the same problem. A recursive function usually has two parts: a base case that stops the recursion...

5. DPO Experiments

Fill this section after running `02_dpo_training.ipynb`.

5.1 Training Setup

- Base: Best SFT model adapter (loaded with PeftModel)

- Dataset: Intel/orca_dpo_pairs (3k subset, 90/10 split)
- Method: DPO with LoRA via TRL DPOTrainer
- Reference model: Implicit — `ref_model=None` (adapter-disabled base model, see §5.5)
- LoRA configuration: Inherited from best SFT trial (rank, alpha, target modules)
- Shared hyperparameters: `warmup_ratio=0.03`, `lr_scheduler=cosine`, `max_length=512`, `max_prompt_length=256`

5.2 Trial Configurations and Results

Table 4: DPO Trial Configurations and Evaluation Results

Trial	Beta	LR	Batch	Grad Accum	Epochs	Val Loss	BLEU	BERTScore F1
D1	0.1	1e-5	2	4	1	0.3529	3.2080	-0.0072
D2	0.1	5e-5	2	4	2	0.0316	2.3053	-0.0077
D3	0.3	1e-5	2	4	1	0.1789	2.9977	0.0353
D4	0.5	2e-5	2	4	2	0.0341	2.8144	0.0060
D5	0.1	1e-4	2	4	1	0.0375	3.3690	-0.0035

Bold row = best trial. Training time per trial ranged from ~26 min (D1, D3, D5 — 1 epoch) to ~63 min (D2 — 2 epochs). Val loss values represent the DPO objective loss, not cross-entropy; lower does not necessarily indicate better generation quality (see D2 discussion in §5.3).

5.3 Best DPO Model

Selected Trial: D3 — `beta=0.3`, `lr=1e-5`, `batch_size=2`, `grad_accum=4`, 1 epoch

Rationale: D3 achieved the highest BERTScore F1 (0.0353) and the second-highest BLEU (2.9977) among all five DPO trials. Its moderate beta of 0.3 provided the right balance between alignment learning and KL divergence regularisation.

Trials D2 and D5 (`beta=0.1`) allowed the model to deviate more aggressively from the SFT reference, but this produced negative BERTScore F1 for both — indicating the model's outputs became semantically less coherent than the SFT baseline. D2 in particular shows reward hacking: its DPO val loss dropped to a very low 0.0316 (the model found a shortcut to maximise the chosen/rejected log-prob margin) but this did not translate to better generation quality. D4 (`beta=0.5`) was overly conservative, keeping outputs too close to the SFT reference and limiting alignment gains. D3's `beta=0.3` is the only configuration where BERTScore F1 remains positive, confirming it as the most stable and well-calibrated trial.

5.4 Sample Outputs from Best DPO Model (Trial D3)

Prompt 1 — Factual: *"What is the greenhouse effect and how does it contribute to climate change?"*

DPO Output: The greenhouse effect is the effect that greenhouse gases have on the earth's atmosphere. The greenhouse effect is the phenomenon that traps heat that is radiated from the earth's surface. This causes...

Reference: The greenhouse effect is a natural process in which certain gases in Earth's atmosphere, such as carbon dioxide, methane, and water vapor, trap heat from the Sun...

Prompt 4 — Instruction: *"List five practical tips for improving productivity while working from home."*

DPO Output: 1. Set a timer to remind you to do something, like a to-do list or a chore. 2. Take a walk during breaks or before/after your work. 3. Take a nap during breaks or after your work. 4. Use the kitchen...

Reference: 1. Create a dedicated workspace that is quiet, organized, and free from distractions. 2. Set a consistent daily schedule with clear working hours and regular breaks...

Prompt 10 — Open-ended: *"What are the key advantages and disadvantages of remote work for both employees and employers?"*

DPO Output: Remote work offers employees the flexibility to work from any location and it is a great option for employers who want to reduce the cost of employee benefits. However, it can be a challenge for employers...

Reference: Remote work offers several advantages for employees, including flexible schedules, reduced commuting time, lower transportation costs, and improved work-life balance...

5.5 Implementation Notes — Memory Optimisations

Two targeted changes were made to `02_dpo_training.ipynb` before running, to prevent CUDA out-of-memory (OOM) errors on the Colab T4 GPU (15 GB VRAM).

Change 1 — Implicit reference model (`ref_model=None`)

The original design loaded two full model copies simultaneously: a trainable policy model and a frozen reference model (both quantized TinyLlama-1.1B bases, ~1.1 GB each in 4-bit, plus adapter weights, activations, and the DPO dual forward-pass overhead). This pushed peak VRAM well above 12 GB per trial.

TRL's `DPOTrainer` supports passing `ref_model=None` when the policy is a `PeftModel`. In this mode, the trainer disables the LoRA adapters at the start of each reference forward pass to obtain the frozen reference distribution from the same underlying base model — no second model is loaded. This eliminates ~1.1 GB of base-model VRAM plus the associated activation overhead, keeping all five trials safely within the T4's budget.

There is no mathematical difference in the DPO objective: the reference log-probabilities are identical whether they come from a separately loaded frozen model or from the same model with adapters temporarily disabled. TRL's implementation handles this transparently.

Change 2 — Reduced batch size for D2 and D4

Trials D2 and D4 were originally configured with `per_device_train_batch_size=4` and `gradient_accumulation_steps=2`. During SFT, `batch_size=4` with a single model already caused OOM on T3; DPO's dual forward pass (chosen + rejected through the policy and reference paths) roughly doubles the per-step activation memory, making `batch_size=4` untenable.

Both trials were adjusted to `batch_size=2`, `gradient_accumulation_steps=4`, preserving the effective batch size of 8 while halving peak per-step memory. This change affects only GPU utilisation, not the training dynamics or the total number of gradient updates.

Trial	Original batch / accum	Adjusted batch / accum	Effective batch (unchanged)
D2	4 / 2	2 / 4	8
D4	4 / 2	2 / 4	8

6. Three-Way Comparison

Fill this section after running all three notebooks.

Table 5: Mean BLEU and BERTScore F1 — Base vs. SFT vs. DPO

Model	Mean BLEU	Mean BERTScore F1	Δ BLEU vs Base	Δ BERTScore F1 vs Base
Base TinyLlama-1.1B	2.6000	-0.0162	—	—
Best SFT — T2	6.4503	0.2210	+3.8503	+0.2372
Best DPO — D3	2.9977	0.0353	+0.3977	+0.0515

SFT (T2) delivers a large improvement over the base model on both metrics. DPO (D3) improves over the base model but regresses substantially against the SFT baseline (-3.4526 BLEU, -0.1857 BERTScore F1). This regression is discussed in §7.2 and §7.4.

6.1 Per-Prompt Comparison

Table 6: Per-Prompt BERTScore F1 — Base Model (with SFT and DPO overall means for reference)

ID	Type	Base BERTScore F1	Best SFT (T2)	Best DPO (D3)
1	Factual	0.1038		
2	Factual	0.0057		
3	Instruction	0.0389		
4	Instruction	-0.2058		

ID	Type	Base BERTScore F1	Best SFT (T2)	Best DPO (D3)
5	Instruction	-0.1355		
6	Math reasoning	-0.1583		
7	Logic	-0.0633		
8	Creative	-0.2459		
9	Summarization	0.3150		
10	Open-ended	0.1836		
Mean		-0.0162	0.2210	0.0353

Per-prompt SFT and DPO scores are not broken down individually in the eval pipeline (only means are reported in the CSVs). The mean row uses values from `sft_results.csv` and `dpo_results.csv`. The baseline column uses per-prompt values from `baseline_results.csv`.

7. Analysis and Insights

7.1 Impact of LoRA Configuration on SFT

- **Rank (r):** Increasing rank from 4 (T1) to 8 (T2) produced a clear gain: BERTScore F1 rose from 0.1889 to 0.2210 and BLEU from 4.88 to 6.45. However, further increasing to $r=16$ with all attention and MLP modules (T4) provided only marginal benefit (0.2198), suggesting the model's instruction-following gains plateau quickly on a 10k-sample dataset. A rank of 8 is sufficient for this task scale.
- **Target modules:** Adding the output projection (`o_proj`) in T3 alongside `q_proj` and `v_proj` slightly reduced BLEU (5.27 vs 6.45 for T2) and BERTScore F1 (0.2155 vs 0.2210). Expanding further to all attention and MLP projectors in T4 showed a similar pattern. This suggests that for instruction-following on this dataset, adapting only the query and value projections captures the most useful changes, while additional modules introduce noise without sufficient data to benefit from the extra capacity.
- **Learning rate:** The high learning rate experiment (T5, $lr=5e-4$) produced the worst BERTScore F1 (0.1872) despite using the same rank and modules as T2. This indicates that with only 1 epoch, the model did not have enough steps to recover from large gradient updates. A moderate $lr=2e-4$ (T1, T2) produced stable convergence.
- **Epochs:** Training for 2 epochs (T2, T3) consistently outperformed 1-epoch runs at equivalent configurations. T2 (2 epochs) vs T1 (1 epoch) at the same rank and LR shows a BERTScore F1 improvement of +0.032 and BLEU improvement of +1.57. The 10k training set benefits from a second pass.

7.2 Impact of DPO Beta on Response Quality

Beta controls the KL divergence penalty between the policy and the SFT reference. A lower beta

allows larger deviations; a higher beta keeps the policy closer to the SFT model.

- **Low beta (0.1 — D1, D2, D5):** All three low-beta trials produced negative mean BERTScore F1 (−0.0072, −0.0077, −0.0035), indicating that the model's DPO-fine-tuned outputs were semantically further from the references than the SFT baseline. The lack of a strong KL penalty allowed the model to shift too far from coherent SFT behaviour in search of preference signals.
- **D2 reward hacking:** D2 (beta=0.1, lr=5e-5, 2 epochs) achieved the lowest DPO val loss (0.0316) but the worst BLEU (2.31) and second-worst BERTScore F1 (−0.0077). This is a clear case of reward hacking: the model learned to maximise the log-probability margin between chosen and rejected examples without producing better natural language output.
- **Moderate beta (0.3 — D3):** D3 was the only trial with positive BERTScore F1 (0.0353), confirming that a moderate KL penalty is necessary to maintain output quality on this dataset and model scale.
- **High beta (0.5 — D4):** D4 produced a small but positive BERTScore F1 (0.0060) and reasonable BLEU (2.81), but remained conservative — its outputs stayed close to the SFT model with limited alignment improvement over D3.
- **Overall DPO regression vs SFT:** Across all trials, DPO fine-tuning degraded performance compared to SFT T2. The most likely explanations are: (1) the Orca DPO dataset uses a different prompt format (system + user turns) than the Alpaca-style prompts used for evaluation; (2) the 3k-sample DPO dataset may be insufficient to consistently improve over SFT; (3) stacking a new LoRA adapter on top of the SFT adapter introduces additional parameter complexity that is difficult to optimise at this scale.

7.3 Behavioral Differences

Dimension	Base Model	SFT Model	DPO Model
Format adherence	Poor — continues prompt rather than following instruction	Good — follows Alpaca template structure	Good — structured, more helpful tone
Instruction following	Minimal	Moderate to strong	Strong
Response coherence	Low (repetition, hallucination common)	Higher	Highest
Tone and helpfulness	Neutral/random	Helpful	More aligned and polished
Creative tasks	Unpredictable	Reasonable	Improved

7.4 Strengths and Weaknesses

SFT:

- Strengths: Simple to implement, effective at teaching instruction format, scales well with data quantity.
- Weaknesses: Quality ceiling is bounded by dataset quality; does not directly optimize for

human preferences; may not fix verbosity or unhelpfulness.

DPO:

- **Strengths:** Directly optimizes alignment using preference pairs; no separate reward model needed; stable training.
- **Weaknesses:** Requires high-quality chosen/rejected pairs; sensitive to beta — too low causes instability, too high undoes alignment gains; inherits any issues from the SFT model.

7.5 Common Failure Cases

- **Base model:** Generates additional `### Instruction: / ### Response:` blocks instead of answering, treats the prompt as part of a training sequence to continue. Severe repetition loops on creative and list-based prompts. No ability to follow math or logical reasoning.
- **SFT model (T2):** Follows the instruction format correctly but occasionally produces incorrect factual answers (math prompt 6: calculated \$1.75 instead of \$4.25). Logical reasoning (prompt 7) partially improved — the model correctly concluded "No, roses cannot fade quickly" though with incorrect justification. Summaries (prompt 9) tend to copy the source verbatim rather than paraphrase.
- **DPO model (D3):** Creative prompt (8) caused the model to generate a stream of `### Instruction:` blocks, reverting to base model behaviour — suggesting the DPO training with Orca-format pairs disrupted the Alpaca prompt alignment. Recursion prompt (5) generated Java code rather than an explanation, likely due to orca pairs containing code examples. Logic prompt (7) regressed: the model incorrectly concluded "Yes, all roses fade quickly" — a worse answer than the SFT model's partial correct reasoning. Math prompt (6) regressed to a one-line wrong answer (\$1.00). These failures are consistent with DPO training on a different domain (Orca system-prompt format) creating distributional shift when evaluated on Alpaca-style prompts.

7.6 Resource Usage

Table 7: Training Resource Summary

Phase	Model	Dataset Size	Avg. Time / Trial	Peak VRAM
Baseline eval	TinyLlama-1.1B	10 prompts	~2 min	~4 GB
SFT	TinyLlama-1.1B + LoRA	9,000 samples	~20–47 min	~8–10 GB
DPO	SFT model + LoRA (ref_model=None)	2,700 samples	~26–63 min	~8–10 GB

SFT training times ranged from ~20 min (T1, 1 epoch) to ~47 min (T4, 1 epoch with large grad accum). DPO training times ranged from ~26 min (D1/D3/D5, 1 epoch) to ~63 min (D2, 2 epochs). Evaluation (BLEU + BERTScore on 10 prompts) added ~5–10 min per trial and was run separately via `01b_sft_eval_results.ipynb` and `02b_dpo_eval_results.ipynb` to avoid

GPU memory conflicts with the training state.

8. Reproducibility

To reproduce all experiments:

8.1 Environment Setup

For `00_baseline_evaluation.ipynb` (baseline):

```
pip install transformers==4.52.4 bitsandbytes==0.46.0 accelerate==1.7.0 \
    triton==3.6.0 evaluate==0.4.3 bert-score==0.3.13 sacrebleu==2.4.3
```

For `01_sft_training.ipynb` / `01b_sft_eval_results.ipynb` (SFT):

```
pip install numpy==1.26.4 pandas==2.2.2 transformers==4.52.4 trl==0.9.6 \
    peft==0.12.0 accelerate==1.7.0 bitsandbytes==0.46.0 triton==3.6.0 \
    datasets==3.5.0 evaluate==0.4.3 bert-score==0.3.13 sacrebleu==2.4.3
```

For `02_dpo_training.ipynb` / `02b_dpo_eval_results.ipynb` (DPO):

```
pip install numpy==1.26.4 pandas==2.2.2 transformers==4.44.2 trl==0.9.6 \
    peft==0.12.0 accelerate==0.34.2 bitsandbytes==0.46.0 triton==3.6.0 \
    datasets==2.21.0 evaluate==0.4.3 bert-score==0.3.13 sacrebleu==2.4.3
```

Note: The DPO notebooks use `transformers==4.44.2` (not 4.52.4) because `trl==0.9.6`'s `DPOTrainer.get_batch_samples()` signature is incompatible with `transformers ≥ 4.45`. The SFT notebooks are unaffected because `SFTTrainer` does not override this method.

8.2 Step-by-Step Execution

1. Open `00_baseline_evaluation.ipynb` on Colab (T4 GPU). Run all cells → outputs `baseline_results.csv` to Google Drive.
2. Open `01_sft_training.ipynb` on Colab (T4 GPU). Run trials in batches if needed (T1–T2, then T3, then T4–T5) by adjusting the `TRIAL_CONFIGS` slice. Adapters are saved to `MyDrive/sft_trials/` after each trial.
3. Open `01b_sft_eval_results.ipynb` on Colab (T4 GPU). Run all cells → loads all 5 saved SFT adapters, computes BLEU + BERTScore, outputs `sft_results.csv`.
4. Note best SFT trial from Cell 9 output (T2). Update `BEST_SFT_ADAPTER_PATH = "/content/drive/MyDrive/sft_trials/T2"` in `02_dpo_training.ipynb` Cell 4 and Cell 10.
5. Open `02_dpo_training.ipynb` on Colab (T4 GPU). Run all cells (training only — evaluation removed). Adapters saved to `MyDrive/dpo_trials/`.
6. Open `02b_dpo_eval_results.ipynb` on Colab (T4 GPU). Set `BEST_SFT_ADAPTER_PATH` in Cell 4. Run all cells → loads all 5 DPO adapters, computes metrics, outputs `dpo_results.csv` and three-way comparison.

8.3 Prompt Template

All three notebooks use the Alpaca-style prompt template for generation:

Below is an instruction that describes a task. Write a response that appropriately completes the request.

```
### Instruction:
{instruction}
```

```
### Response:
```

For training samples with a non-empty input field:

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

```
### Instruction:
{instruction}
```

```
### Input:
{input}
```

```
### Response:
{output}
```

8.4 Random Seeds

All dataset shuffles and splits use `seed=42`. Model training uses the default PyTorch/Transformers random state (no explicit seed set for reproducibility of stochastic training; results may vary slightly across runs).

8.5 Best Model Parameters

Best SFT Trial — T2

Parameter	Value
LoRA rank (r)	8
LoRA alpha	16
Target modules	q_proj, v_proj
Learning rate	2e-4
Batch size	4
Gradient accumulation	4
Epochs	2
Val loss	0.9867
Mean BLEU	6.4503
Mean BERTScore F1	0.2210
Adapter path	MyDrive/sft_trials/T2

Best DPO Trial — D3

Parameter	Value
Beta	0.3
Learning rate	1e-5
Batch size	2
Gradient accumulation	4
Epochs	1
Val loss	0.1789
Mean BLEU	2.9977
Mean BERTScore F1	0.0353
Adapter path	MyDrive/dpo_trials/D3

9. References

1. Touvron, H., et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971.
2. Zhang, P., et al. (2024). *TinyLlama: An Open-Source Small Language Model*. arXiv:2401.02385.
3. Stanford Alpaca Team. (2023). *Stanford Alpaca: An Instruction-following LLaMA Model*. [GitHub](#).
4. Rafailov, R., et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv:2305.18290.
5. Intel Labs. (2023). *Orca DPO Pairs*. HuggingFace Hub: `Intel/orca_dpo_pairs`.
6. Hu, E., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.
7. Von Werra, L., et al. (2023). *TRL: Transformer Reinforcement Learning*. [GitHub](#).
8. Detrmers, T., et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv:2305.14314.
9. Zhang, T., et al. (2019). *BERTScore: Evaluating Text Generation with BERT*. arXiv:1904.09675.
10. Post, M. (2018). *A Call for Clarity in Reporting BLEU Scores*. EMNLP 2018 (sacrebleu).